

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO
TOCANTINS**

CAMPUS PALMAS

CURSO TÉCNICO/TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

NTPB DADOS

**Dashboard Gerencial de Vendas com Arquitetura Full-Stack e Visualização
em Tempo Real**

Wanderson | Gabriel Victor | Rafael | Elias | Alessandra

Palmas, Tocantins

2026

Resumo

O presente trabalho documenta o desenvolvimento do sistema NTPB Dados, uma solução de Business Intelligence (BI) e gestão transacional voltada para o segmento de comércio eletrônico. O projeto foi concebido como dashboard gerencial de vendas com arquitetura full-stack, tendo como objetivo centralizar o controle de inventário e fornecer visualizações analíticas em tempo real que apoiem a tomada de decisão gerencial.

A solução integra tecnologias modernas de desenvolvimento web: React com TypeScript no front-end, Node.js com Express no back-end e PostgreSQL como sistema gerenciador de banco de dados relacional. A visualização de dados é implementada com a biblioteca Chart.js, permitindo a geração dinâmica de gráficos de barras, linhas e indicadores de desempenho (KPIs). A comunicação entre as camadas é realizada por meio de uma API REST, seguindo os princípios arquiteturais de separação de responsabilidades.

Os resultados esperados incluem a redução do tempo de análise de desempenho comercial, a automação do controle de estoque em tempo real e a geração de relatórios consolidados por período e categoria. O projeto aplica, de forma integrada, conceitos de banco de dados relacionais, operações CRUD, agregações SQL, versionamento com Conventional Commits e boas práticas de engenharia de software.

Palavras-chave: Business Intelligence. Dashboard. Full-Stack. PostgreSQL. React. TypeScript. API REST.

1 Introdução

1.1 Contextualização do Problema

O crescimento acelerado do comércio eletrônico nas últimas décadas transformou profundamente a forma como empresas gerenciam suas operações comerciais. Segundo dados do relatório Webshoppers da Ebit/Nielsen, o e-commerce brasileiro registrou crescimento contínuo de dois dígitos ao longo dos anos 2010, fenômeno intensificado pela pandemia de COVID-19 a partir de 2020. Nesse contexto, a capacidade de coletar, processar e interpretar dados de vendas tornou-se um diferencial competitivo fundamental.

Pequenas e médias empresas, contudo, frequentemente operam sem ferramentas adequadas de análise de desempenho. O controle de vendas é realizado, em muitos casos, por meio de planilhas eletrônicas estáticas, que não permitem consultas dinâmicas, cruzamento de dados em tempo real ou geração automatizada de relatórios. Essa limitação compromete a agilidade na identificação de tendências, no monitoramento de estoque e na tomada de decisões baseadas em evidências.

1.2 Importância da Análise de Vendas

A análise de vendas constitui um dos pilares da gestão empresarial moderna. Ela permite identificar quais produtos apresentam maior demanda, em que períodos do ano as vendas se concentram, quais categorias contribuem de forma mais expressiva para o faturamento e onde se encontram os gargalos operacionais. Sistemas de informação dedicados a essa finalidade, como os dashboards gerenciais, traduzem volumes expressivos de dados transacionais em representações visuais intuitivas, tornando o processo analítico mais eficiente e acessível a gestores sem formação técnica em ciência de dados.

1.3 Justificativa do Projeto

O projeto NTPB Dados surge como resposta a essa demanda, propondo o desenvolvimento de um sistema web que simule o ambiente operacional de uma loja virtual e disponibilize um painel de controle com indicadores-chave de desempenho (KPIs), gráficos interativos e relatórios filtráveis. A escolha por tecnologias de ampla adoção no mercado — React, Node.js, PostgreSQL e Chart.js — garante ao projeto relevância acadêmica e aplicabilidade profissional imediata.

Do ponto de vista acadêmico, o projeto integra disciplinas fundamentais do curso de Análise e Desenvolvimento de Sistemas: banco de dados relacionais, desenvolvimento web, engenharia de requisitos, testes de software e arquitetura de sistemas. A execução em equipe multidisciplinar de cinco membros replica dinâmicas reais de desenvolvimento colaborativo, com uso de controle de versão e metodologias ágeis.

1.4 Delimitação do Escopo

O escopo do presente projeto abrange o desenvolvimento de um sistema web funcional com as seguintes características: cadastro e gestão de produtos e categorias, registro de vendas com controle automático de estoque, dashboard analítico com gráficos e KPIs, filtros por período e categoria, painel administrativo e

documentação técnica completa. O sistema simula uma loja virtual fictícia, não contemplando integração com gateways de pagamento reais, módulos de logística ou autenticação OAuth com provedores externos.

2 Fundamentação Teórica

2.1 Sistemas de Informação

Sistemas de Informação (SI) são conjuntos organizados de pessoas, procedimentos, software, hardware e dados que coletam, transformam e distribuem informações em uma organização. Laudon e Laudon (2014) os definem como componentes inter-relacionados que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisões, a coordenação, o controle, a análise e a visualização em uma organização. Os SI transacionais, como o NTPB Dados, têm como função primária o registro de transações operacionais — neste caso, vendas de produtos — constituindo a base de dados sobre a qual os sistemas analíticos operam.

2.2 Business Intelligence

Business Intelligence (BI) é o conjunto de estratégias, processos, aplicações e tecnologias utilizadas para coletar, integrar, analisar e apresentar informações de negócios com o objetivo de suportar decisões empresariais. Conforme Turban et al. (2011), BI transforma dados brutos em informações acionáveis, por meio de processos de extração, transformação e carga (ETL), data warehousing e ferramentas de análise multidimensional. Dashboards gerenciais são considerados a interface mais direta de BI, por condensar métricas críticas em uma única tela de visualização.

2.3 Dashboards Gerenciais

Um dashboard gerencial é uma representação visual consolidada de informações-chave de desempenho, projetada para ser percebida de forma imediata. Few (2006) distingue dashboards operacionais, táticos e estratégicos conforme o perfil do usuário e o horizonte temporal das informações apresentadas. No contexto do NTPB Dados, o dashboard é de natureza operacional e tática: fornece dados em tempo real sobre vendas recentes e permite análises históricas por período. A eficácia de um dashboard está diretamente relacionada à escolha adequada dos tipos de gráfico, à hierarquia visual e à minimização de ruído informacional.

2.4 Banco de Dados Relacionais

O modelo relacional, proposto por Edgar F. Codd em 1970, organiza os dados em tabelas (relações) compostas por linhas (tuplas) e colunas (atributos). Cada tabela representa uma entidade do domínio do problema, e os relacionamentos entre entidades são expressos por meio de chaves estrangeiras. Sistemas Gerenciadores de Banco de Dados Relacionais (SGBDRs) como o PostgreSQL implementam este modelo com garantias de integridade fornecidas pelas propriedades ACID: Atomicidade, Consistência, Isolamento e Durabilidade. No NTPB Dados, o PostgreSQL é utilizado para armazenar o catálogo de produtos, o histórico de vendas e os dados de categorias, com restrições de integridade referencial aplicadas por chaves estrangeiras.

2.5 SQL — Structured Query Language

A Structured Query Language (SQL) é a linguagem padrão para definição, manipulação e consulta de dados em SGBDRs. Silberschatz et al. (2020) dividem os comandos SQL em quatro sublinguagens: DDL (Data Definition Language) para criação de estruturas, DML (Data Manipulation Language) para inserção, atualização e remoção de registros, DQL (Data Query Language) para consultas e DCL (Data Control Language) para controle de acesso. O projeto utiliza extensivamente funções de agregação — SUM(), COUNT(), AVG() — em conjunto com cláusulas GROUP BY e filtros temporais para gerar os indicadores exibidos no dashboard.

2.6 APIs REST

Representational State Transfer (REST) é um estilo arquitetural proposto por Roy Fielding em sua tese doutoral de 2000, que define um conjunto de restrições para o desenvolvimento de serviços web escaláveis e de baixo acoplamento. Uma API RESTful utiliza os métodos HTTP (GET, POST, PUT, DELETE) para realizar operações sobre recursos identificados por URIs, retornando representações em formato JSON ou XML. No NTPB Dados, a API REST é implementada com Node.js e Express, expondo endpoints para gerenciamento de produtos, categorias, vendas e agregações analíticas.

2.7 Visualização de Dados

A visualização de dados é a representação gráfica de informações com o objetivo de facilitar a compreensão de padrões, tendências e outliers. Munzner (2014) propõe um modelo aninhado de design de visualizações que considera domínio, abstração de dados, idiomas visuais e algoritmos. A biblioteca Chart.js, utilizada no projeto, implementa os principais idiomas visuais em ambiente web: gráficos de barras para comparação categórica, gráficos de linha para séries temporais, gráficos de rosca (doughnut) para distribuição proporcional e scorecards para KPIs numéricos.

2.8 Indicadores de Desempenho (KPIs)

Key Performance Indicators (KPIs) são métricas quantificáveis que refletem o grau de sucesso de uma organização no alcance de seus objetivos. Parmenter (2015) distingue indicadores de resultado (lagging indicators) dos indicadores de processo (leading indicators). No contexto de um dashboard de vendas, os KPIs primários incluem receita total, número de transações, ticket médio e volume de produtos vendidos. Esses indicadores fundamentam as seções analíticas do NTPB Dados e são calculados por consultas SQL de agregação executadas em tempo real.

3 Objetivos

3.1 Objetivo Geral

Desenvolver um sistema web de análise e gestão de vendas fictícias, denominado NTPB Dados, capaz de simular o ambiente operacional de uma loja virtual e fornecer visualizações analíticas em tempo real por meio de um dashboard gerencial interativo.

3.2 Objetivos Específicos

- Modelar e implementar um banco de dados relacional em PostgreSQL com entidades para produtos, categorias, vendas e itens de venda, aplicando restrições de integridade referencial.
- Desenvolver uma API REST com Node.js e Express para expor operações CRUD sobre as entidades do sistema e consultas analíticas de agregação.
- Implementar a interface de usuário com React e TypeScript, garantindo responsividade, consistência visual por Design System e usabilidade adequada ao perfil gerencial.
- Integrar a biblioteca Chart.js para renderização de gráficos de vendas por período, por categoria, ranking de produtos e faturamento mensal.
- Implementar a lógica de abatimento automático de estoque na confirmação de vendas, com validação de disponibilidade prévia.
- Documentar o projeto em conformidade com padrões acadêmicos, incluindo diagrama ER, dicionário de dados, especificação de API e plano de testes.
- Aplicar metodologia de versionamento com Conventional Commits ao longo de todo o ciclo de desenvolvimento.

4 Levantamento de Requisitos

4.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve prover, expressas sob a perspectiva do comportamento esperado em resposta a entradas específicas.

Código	Nome	Descrição	Prioridade
RF-001	Cadastrar Categoria	O sistema deve permitir criar, listar, editar e excluir categorias de produtos, com campos de nome e descrição.	Alta
RF-002	Cadastrar Produto	O sistema deve permitir criar, listar, editar e excluir produtos, associando-os a uma categoria e registrando nome, preço e quantidade em estoque.	Alta
RF-003	Registrar Venda	O sistema deve permitir registrar uma venda vinculando um ou mais produtos com suas respectivas quantidades, gerando automaticamente o total da transação.	Alta
RF-004	Abater Estoque	Ao confirmar uma venda, o sistema deve decrementar a quantidade em estoque de cada produto envolvido. A operação deve ser atômica.	Alta
RF-005	Validar Disponibilidade	O sistema deve impedir o registro de uma venda quando a quantidade solicitada de qualquer produto exceder o estoque disponível, exibindo mensagem de erro clara.	Alta
RF-006	Exibir Dashboard	O sistema deve exibir um painel com KPIs (Receita Total, Total de Vendas, Ticket Médio, Produtos Vendidos) calculados em tempo real.	Alta
RF-007	Gráfico por Período	O sistema deve exibir gráfico de linha representando o volume de vendas ao longo do tempo, com filtro de intervalo de datas.	Alta
RF-008	Gráfico por Categoria	O sistema deve exibir gráfico de barras ou rosca representando a distribuição de vendas por categoria de produto.	Média
RF-009	Ranking de Produtos	O sistema deve exibir lista dos produtos mais vendidos, ordenada por quantidade total vendida, com opção de filtro por período.	Média
RF-010	Filtrar por	O sistema deve permitir filtrar todos os	Alta

Código	Nome	Descrição	Prioridade
	Período	indicadores e gráficos por intervalo de datas definido pelo usuário.	
RF-011	Filtrar por Categoria	O sistema deve permitir filtrar os dados exibidos no dashboard e relatórios por categoria de produto.	Média
RF-012	Relatório de Vendas	O sistema deve gerar relatório consolidado de vendas com detalhamento por transação, exibindo data, produtos, quantidades e valores.	Média
RF-013	Painel Administrativo	O sistema deve disponibilizar interface administrativa para gerenciamento centralizado de produtos, categorias e consulta de histórico de vendas.	Alta

4.2 Critérios de Aceitação — Requisitos Prioritários

RF-003 — Registrar Venda

- Dado que o usuário seleciona produtos e quantidades válidas e confirma a venda, o sistema deve persistir a venda e seus itens e decrementar o estoque em menos de 200ms.
- O sistema deve retornar código HTTP 201 com o objeto da venda criada na resposta.

RF-005 — Validar Disponibilidade

- Dado que o usuário tenta registrar uma venda com quantidade maior que o estoque disponível, o sistema deve retornar código HTTP 422 com mensagem identificando o produto indisponível.
- O estoque não deve ser alterado em caso de validação negativa.

4.3 Requisitos Não Funcionais

Código	Categoria	Descrição
RNF-001	Usabilidade	A interface deve ser responsiva, adaptando-se a resoluções a partir de 768px (tablet) e 1024px (desktop), seguindo padrões de design SaaS.
RNF-002	Manutenibilidade	O sistema deve ser desenvolvido integralmente em TypeScript para garantir segurança de tipos, reduzindo erros de integração entre front-end e back-end.
RNF-003	Performance	O tempo de resposta para atualização dos gráficos após o registro de uma venda não deve exceder 200ms em condições normais de operação.

Código	Categoria	Descrição
RNF-004	Integridade de Dados	O banco de dados PostgreSQL deve implementar restrições de chave estrangeira, unicidade e NOT NULL para garantir consistência dos dados transacionais.
RNF-005	Escalabilidade	A arquitetura em camadas deve permitir a substituição independente de qualquer componente (front-end, back-end ou SGBD) sem refatoração global.
RNF-006	Disponibilidade	O sistema deve estar disponível para uso em ambiente local de desenvolvimento, com tempo de inicialização inferior a 30 segundos.
RNF-007	Segurança	Todas as entradas de usuário devem ser validadas no back-end antes da persistência, prevenindo injeção SQL e dados inconsistentes.

5 Regras de Negócio

5.1 Cadastro de Produtos

- RN-001: O preço de um produto deve ser um valor decimal positivo maior que zero, com no máximo duas casas decimais.
- RN-002: A quantidade em estoque não pode ser negativa. O valor mínimo permitido é zero.
- RN-003: Todo produto deve estar vinculado a exatamente uma categoria existente no sistema. Não é permitido cadastrar produtos sem categoria.
- RN-004: O nome do produto deve ser único dentro da mesma categoria.

5.2 Registro de Vendas

- RN-005: Uma venda deve conter pelo menos um item (produto e quantidade).
- RN-006: A quantidade solicitada de cada produto em uma venda deve ser um inteiro positivo maior que zero.
- RN-007: O sistema deve verificar a disponibilidade de estoque de todos os itens antes de confirmar a venda. Se qualquer item estiver indisponível, a venda inteira deve ser rejeitada.
- RN-008: O valor total da venda é calculado automaticamente pelo sistema como a soma dos produtos entre preço unitário e quantidade de cada item.
- RN-009: O preço unitário registrado no item de venda deve ser o preço praticado no momento da transação, garantindo histórico imutável mesmo que o preço do produto seja alterado posteriormente.
- RN-010: O abatimento de estoque e a persistência da venda devem ocorrer de forma atômica: ou ambas as operações são concluídas, ou nenhuma é aplicada.

5.3 Relatórios e Indicadores

- RN-011: Os KPIs exibidos no dashboard devem ser calculados com base nas vendas registradas no período filtrado, ou na totalidade do histórico quando nenhum filtro estiver ativo.
- RN-012: O Ticket Médio é calculado como a razão entre a Receita Total e o número de transações (vendas) no período.
- RN-013: O Ranking de Produtos Mais Vendidos deve ser ordenado pela quantidade total de itens vendidos no período, não pelo faturamento gerado.
- RN-014: Produtos com estoque igual a zero devem ser destacados visualmente no painel administrativo.

6 Arquitetura do Sistema

6.1 Visão Geral — Arquitetura em Três Camadas

O NTPB Dados adota a arquitetura em três camadas (Three-Tier Architecture), que separa as responsabilidades de apresentação, lógica de negócio e persistência de dados em módulos independentes. Essa separação favorece a manutenibilidade, a testabilidade e a escalabilidade horizontal de cada camada.

Camada	Tecnologia	Responsabilidade
Apresentação (Front-end)	React + TypeScript + Chart.js	Renderização da interface, navegação, consumo da API REST e visualização de dados.
Lógica de Negócio (Back-end)	Node.js + Express + TypeScript	Processamento de requisições HTTP, validação, regras de negócio, orquestração de transações e comunicação com o banco.
Dados (SGBD)	PostgreSQL	Persistência, integridade referencial, consultas analíticas de agregação e controle transacional ACID.

6.2 Fluxo de Comunicação

O front-end comunica-se exclusivamente com o back-end por meio de requisições HTTP à API REST. O back-end, por sua vez, interage com o PostgreSQL via driver de banco de dados (pg ou ORM equivalente). Não há comunicação direta entre o front-end e o banco de dados. Essa separação garante que toda a lógica de validação e segurança seja processada no servidor, protegendo a integridade dos dados.

O fluxo de uma operação típica de venda percorre as seguintes etapas: (1) o usuário seleciona os produtos na interface React; (2) o front-end envia uma requisição POST para /api/sales; (3) o back-end valida os dados, verifica o estoque e executa a transação SQL; (4) o PostgreSQL confirma a operação; (5) o back-end retorna a resposta HTTP 201 com os dados da venda criada; (6) o front-end atualiza o dashboard com os novos indicadores.

6.3 Estrutura de Diretórios do Projeto

Diretório/Arquivo	Descrição
projeto-NTPBDados/	Raiz do repositório
backend/	Aplicação Node.js + Express
src/routes/	Definição dos endpoints REST
src/controllers/	Lógica de tratamento das requisições

Diretório/Arquivo	Descrição
src/services/	Regras de negócio e acesso ao banco
src/models/	Definições de tipos TypeScript
src/database/	Configuração da conexão e migrations
frontend/	Aplicação React + TypeScript
src/components/	Componentes reutilizáveis de interface
src/pages/	Páginas principais do sistema
src/services/	Módulos de chamada à API
src/types/	Interfaces e tipos TypeScript
docs/	Documentação técnica do projeto

7 Modelagem do Sistema

7.1 Diagrama de Casos de Uso

7.1.1 Atores

- Gerente/Administrador: responsável pelo cadastro de produtos e categorias, registro de vendas e consulta de relatórios e indicadores no dashboard.
- Sistema: ator secundário que executa automaticamente o abatimento de estoque após confirmação de vendas e o cálculo de KPIs.

7.1.2 Casos de Uso Identificados

Identificador	Caso de Uso	Ator Principal	Descrição Resumida
UC-01	Gerenciar Categorias	Administrador	CRUD de categorias de produtos (criar, listar, editar, excluir).
UC-02	Gerenciar Produtos	Administrador	CRUD de produtos com vinculação à categoria, preço e estoque.
UC-03	Registrar Venda	Administrador	Seleção de produtos, inserção de quantidades e confirmação da transação.
UC-04	Validar Estoque	Sistema	Verificação automática da disponibilidade antes de confirmar venda.
UC-05	Abater Estoque	Sistema	Decremento automático do estoque após confirmação da venda.
UC-06	Visualizar Dashboard	Administrador	Exibição de KPIs, gráficos e ranking em tempo real.
UC-07	Aplicar Filtros	Administrador	Filtragem dos dados exibidos por período e/ou categoria.
UC-08	Consultar Relatórios	Administrador	Geração de relatório detalhado de vendas com itens e valores.

7.2 Modelo Entidade-Relacionamento

7.2.1 Entidades e Atributos

O modelo de dados do NTPB Dados é composto por cinco entidades principais, cujas definições são apresentadas a seguir.

Entidade CATEGORIA: representa as classificações dos produtos comercializados. Atributos: id (PK), nome (NOT NULL, UNIQUE), descricao.

Entidade PRODUTO: representa os itens disponíveis para venda. Atributos: id (PK), nome (NOT NULL), preco (DECIMAL, NOT NULL), estoque (INTEGER, NOT NULL, >= 0), categoria_id (FK -> CATEGORIA).

Entidade VENDA: representa uma transação de compra. Atributos: id (PK), data (TIMESTAMP NOT NULL, DEFAULT NOW()), total (DECIMAL, NOT NULL).

Entidade ITEM_VENDA: representa cada produto dentro de uma venda (entidade associativa). Atributos: id (PK), venda_id (FK -> VENDA), produto_id (FK -> PRODUTO), quantidade (INTEGER, NOT NULL, > 0), preco_unitario (DECIMAL, NOT NULL).

Entidade USUARIO: representa os usuários com acesso ao painel administrativo. Atributos: id (PK), nome (NOT NULL), email (NOT NULL, UNIQUE), senha_hash (NOT NULL), perfil (VARCHAR: 'admin' | 'operador'), criado_em (TIMESTAMP).

7.2.2 Relacionamentos e Cardinalidades

- CATEGORIA — PRODUTO: Um para Muitos (1:N). Uma categoria pode classificar múltiplos produtos; cada produto pertence a exatamente uma categoria.
- VENDA — ITEM_VENDA: Um para Muitos (1:N). Uma venda possui um ou mais itens; cada item pertence a exatamente uma venda.
- PRODUTO — ITEM_VENDA: Um para Muitos (1:N). Um produto pode compor itens de múltiplas vendas; cada item referencia exatamente um produto.

8 Modelagem do Banco de Dados

8.1 Dicionário de Dados

Tabela	Atributo	Tipo	Restrições	Descrição
categoria	id	SERIAL	PK	Identificador único auto-incrementado.
categoria	nome	VARCHAR(100)	NOT NULL, UNIQUE	Nome da categoria. Deve ser único no sistema.
categoria	descricao	TEXT	—	Descrição opcional da categoria.
produto	id	SERIAL	PK	Identificador único auto-incrementado.
produto	nome	VARCHAR(200)	NOT NULL	Nome do produto comercializado.
produto	preco	DECIMAL(10, 2)	NOT NULL, > 0	Preço de venda unitário com duas casas decimais.
produto	estoque	INTEGER	NOT NULL, >= 0	Quantidade disponível. Decrementado a cada venda.
produto	categoria_id	INTEGER	FK -> categoria	Referência à categoria do produto.
venda	id	SERIAL	PK	Identificador único da transação.
venda	data	TIMESTAMP	NOT NULL, DEFAULT NOW()	Data e hora UTC do registro da venda.
venda	total	DECIMAL(12, 2)	NOT NULL	Valor total calculado da venda.
item_venda	id	SERIAL	PK	Identificador único do item.
item_venda	venda_id	INTEGER	FK -> venda	Referência à venda que contém este item.
item_venda	produto_id	INTEGER	FK -> produto	Referência ao produto vendido.
item_venda	quantidade	INTEGER	NOT NULL, > 0	Quantidade de unidades do produto neste item.
item_venda	preco_unitario	DECIMAL(10, 2)	NOT NULL	Preço praticado no momento da venda (imutável).
usuario	id	SERIAL	PK	Identificador único do usuário.

Tabela	Atributo	Tipo	Restrições	Descrição
usuario	nome	VARCHAR(150)	NOT NULL	Nome completo do usuário.
usuario	email	VARCHAR(200)	NOT NULL, UNIQUE	E-mail de acesso, único por usuário.
usuario	senha_hash	VARCHAR(255)	NOT NULL	Hash bcrypt da senha de acesso.
usuario	perfil	VARCHAR(20)	NOT NULL	Perfil de acesso: 'admin' ou 'operador'.
usuario	criado_em	TIMESTAMP	DEFAULT NOW()	Data de criação do registro.

8.2 Comandos DDL de Criação das Tabelas

Os comandos SQL abaixo representam a estrutura do banco de dados em PostgreSQL:

```
CREATE TABLE categoria (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(100) NOT NULL UNIQUE,
  descricao TEXT
);

CREATE TABLE produto (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(200) NOT NULL,
  preco DECIMAL(10,2) NOT NULL CHECK (preco > 0),
  estoque INTEGER NOT NULL DEFAULT 0 CHECK (estoque >= 0),
  categoria_id INTEGER NOT NULL REFERENCES categoria(id)
);

CREATE TABLE venda (
  id SERIAL PRIMARY KEY,
  data TIMESTAMP NOT NULL DEFAULT NOW(),
  total DECIMAL(12,2) NOT NULL
);

CREATE TABLE item_venda (
  id SERIAL PRIMARY KEY,
  venda_id INTEGER NOT NULL REFERENCES venda(id),
  produto_id INTEGER NOT NULL REFERENCES produto(id),
  quantidade INTEGER NOT NULL CHECK (quantidade > 0),
  preco_unitario DECIMAL(10,2) NOT NULL
);
```

9 Especificação da API REST

9.1 Produtos

Método	Endpoint	Descrição	Resposta
GET	/api/products	Retorna lista de todos os produtos com estoque atual.	200 OK — Array de objetos produto.
GET	/api/products/:id	Retorna dados de um produto específico.	200 OK 404 Not Found
POST	/api/products	Cria novo produto. Body: { nome, preco, estoque, categoria_id }.	201 Created — Objeto criado.
PUT	/api/products/:id	Atualiza dados de um produto existente.	200 OK — Objeto atualizado.
DELETE	/api/products/:id	Remove produto. Falha se houver vendas vinculadas.	204 No Content 409 Conflict

9.2 Categorias

Método	Endpoint	Descrição	Resposta
GET	/api/categories	Retorna lista de categorias com contagem de produtos.	200 OK
POST	/api/categories	Cria nova categoria. Body: { nome, descricao }.	201 Created
PUT	/api/categories/:id	Atualiza nome ou descrição da categoria.	200 OK 404 Not Found
DELETE	/api/categories/:id	Remove categoria sem produtos vinculados.	204 No Content 409 Conflict

9.3 Vendas

Método	Endpoint	Descrição	Resposta
GET	/api/sales	Lista vendas com filtros opcionais: ? start=&end=&categoria=.	200 OK — Array de vendas.
GET	/api/sales/:id	Retorna venda com todos os itens.	200 OK 404 Not Found
POST	/api/sales	Registra venda. Body: { itens:	201 Created 422

Método	Endpoint	Descrição	Resposta
		[[produto_id, quantidade]].	Estoque insuficiente

9.4 Dashboard e Relatórios

Método	Endpoint	Descrição	Resposta
GET	/api/dashboard/stats	Retorna KPIs: receita_total, total_vendas, ticket_medio, produtos_vendidos.	200 OK — Objeto de agregados.
GET	/api/dashboard/sales-by-period	Séries temporais para gráfico de linha. Params: ? start=&end=&granularity=day week month.	200 OK — Array de { data, total }.
GET	/api/dashboard/sales-by-category	Distribuição de vendas por categoria para gráfico de barras/rosca.	200 OK — Array de { categoria, total }.
GET	/api/dashboard/top-products	Ranking dos N produtos mais vendidos por quantidade. Param: ?limit=10.	200 OK — Array ranqueado.

9.5 Exemplo de Requisição e Resposta — POST /api/sales

```
// Requisição
POST /api/sales Content-Type: application/json
{
  "itens": [
    { "produto_id": 3, "quantidade": 2 },
    { "produto_id": 7, "quantidade": 1 }
  ]
}

// Resposta – 201 Created
{
  "id": 42,
  "data": "2026-06-09T14:35:00Z",
  "total": 189.90,
  "itens": [ ... ]
}
```

10 Dashboard e Indicadores

10.1 Gráficos Implementados

Vendas por Período (Gráfico de Linha)

Representa a evolução do volume de vendas ao longo do tempo, permitindo identificar tendências, sazonalidades e picos de demanda. O eixo horizontal exibe a dimensão temporal (dia, semana ou mês conforme granularidade selecionada) e o eixo vertical exibe o valor total de vendas. Dados obtidos via GET `/api/dashboard/sales-by-period`.

Vendas por Categoria (Gráfico de Barras)

Compara o desempenho comercial das diferentes categorias de produtos no período filtrado. Barras verticais permitem comparação direta entre categorias. Uma variante em rosca (doughnut) exibe a distribuição proporcional. Dados obtidos via GET `/api/dashboard/sales-by-category`.

Ranking de Produtos Mais Vendidos

Lista os produtos ordenados pela quantidade total de unidades vendidas no período. O componente exibe nome do produto, categoria, quantidade vendida e receita gerada. Dados obtidos via GET `/api/dashboard/top-products` com limit configurável.

Faturamento Mensal (Gráfico de Barras Agrupadas)

Apresenta comparativo de faturamento entre os meses do ano corrente e do ano anterior, quando disponível. Permite ao gestor avaliar o crescimento ou retração nas vendas em perspectiva anual.

10.2 Indicadores de Desempenho (KPIs)

KPI	Fórmula de Cálculo	Descrição
Receita Total	<code>SUM(venda.total) WHERE data IN período</code>	Soma de todos os valores de vendas no período filtrado.
Total de Vendas	<code>COUNT(venda.id) WHERE data IN período</code>	Número de transações registradas no período.
Ticket Médio	<code>Receita Total / Total de Vendas</code>	Valor médio por transação. Indica o padrão de compra do cliente.
Produtos Vendidos	<code>SUM(item_venda.quantidade) WHERE venda IN período</code>	Quantidade total de unidades comercializadas no período.

11 Plano de Testes

11.1 Estratégia de Testes

O plano de testes do NTPB Dados contempla três níveis de verificação: testes unitários para serviços e utilitários do back-end, testes de integração para os endpoints da API REST e testes de interface para os componentes React críticos. A estratégia prioriza os fluxos de negócio de maior risco: registro de vendas e controle de estoque.

11.2 Casos de Teste

ID	Caso de Teste	Pré-Condição	Passos	Resultado Esperado
CT-001	Registrar venda com estoque disponível	Produto A com estoque = 10	POST /api/sales com { produto_id: A, quantidade: 3 }	HTTP 201. Estoque de A = 7. Venda registrada no banco.
CT-002	Bloquear venda com estoque insuficiente	Produto B com estoque = 2	POST /api/sales com { produto_id: B, quantidade: 5 }	HTTP 422. Estoque de B inalterado. Venda não registrada.
CT-003	Cadastrar produto sem categoria	Categoria inexistente no sistema	POST /api/products sem campo categoria_id	HTTP 422. Mensagem de validação retornada.
CT-004	Calcular KPIs corretamente	3 vendas de R\$100, R\$200, R\$150	GET /api/dashboard/stats	receita_total=450, total_vendas=3, ticket_medio=150.
CT-005	Filtrar dashboard por período	Vendas em jan e fev de 2026	GET /api/dashboard/stats?start=2026-01-01&end=2026-01-31	Apenas as vendas de janeiro consideradas nos KPIs.
CT-006	Excluir categoria com produto vinculado	Categoria com 2 produtos	DELETE /api/categories/:id	HTTP 409. Mensagem de conflito. Categoria preservada.
CT-007	Exibir ranking de produtos	5 produtos com vendas distintas	GET /api/dashboard/top-products?limit=3	Retorna exatamente os 3 produtos com maior quantidade vendida.
CT-008	Atomicidade da venda com múltiplos itens	Item 1 disponível, Item 2 sem estoque	POST /api/sales com ambos os itens	HTTP 422. Nenhum estoque alterado. Nenhuma venda persistida.

12 Cronograma de Desenvolvimento

Fase	Atividade	Responsáveis	Semanas	Status
1 — Planejamento	Levantamento de requisitos e definição do escopo	Equipe	1–2	Concluído
2 — Modelagem	Diagrama ER, dicionário de dados e wireframes	Wanderson, Alessandra	2–3	Concluído
3 — Back-end	Configuração do ambiente, modelos e rotas básicas	Wanderson, Rafael	3–5	Concluído
3 — Back-end	Lógica de venda e abatimento de estoque	Wanderson, Rafael	5–6	Concluído
4 — Front-end	Estrutura React, Design System e componentes base	Gabriel Victor, Elias	4–6	Concluído
4 — Front-end	Integração com API, gráficos Chart.js e KPIs	Gabriel Victor, Elias	6–8	Em andamento
5 — Integração	Integração completa front-end e back-end	Equipe	8–9	Pendente
6 — Testes	Execução do plano de testes e correção de falhas	Equipe	9–10	Pendente
7 — Documentação	Documentação técnica final e revisão	Wanderson, Alessandra	10–11	Em andamento
8 — Entrega	Apresentação e entrega do projeto	Equipe	11–12	Pendente

13 Análise de Riscos

13.1 Riscos Identificados

ID	Risco	Tipo	Probabilidade	Impacto	Estratégia de Mitigação
R-001	Condições de corrida no abatimento de estoque em requisições simultâneas	Técnico	Média	Alto	Utilizar transações SQL com nível de isolamento <code>SERIALIZABLE</code> e locks seletivos.
R-002	Divergência de versões de dependências npm entre membros da equipe	Técnico	Alta	Médio	Fixar versões exatas no <code>package.json</code> e usar arquivo <code>.nvmrc</code> com versão do Node.js.
R-003	Escopo crescente (scope creep) durante o desenvolvimento	Operacional	Média	Alto	Documentar formalmente o escopo e recusar alterações não previstas sem aprovação.
R-004	Atraso na integração front-end e back-end por desalinhamento de contratos da API	Técnico	Média	Alto	Definir e documentar os contratos da API antes do início do desenvolvimento paralelo.
R-005	Ausência de membros da equipe em fases críticas	Operacional	Baixa	Alto	Documentar todas as decisões técnicas e garantir que mais de um membro conheça cada módulo.
R-006	Inconsistência de dados por falha em transações de venda	Técnico	Baixa	Crítico	Implementar rollback explícito em todas as transações e logs de auditoria para operações críticas.

14 Resultados Esperados

14.1 Benefícios Técnicos

Do ponto de vista técnico, o projeto NTPB Dados proporciona a implementação prática e integrada de um conjunto amplo de competências do curso de ADS: modelagem relacional de banco de dados, desenvolvimento de APIs REST, construção de interfaces reativas com componentes TypeScript e integração de bibliotecas de visualização de dados. A aplicação das propriedades ACID nas transações de venda demonstra, em ambiente real de desenvolvimento, os fundamentos teóricos estudados na disciplina de Banco de Dados.

A adoção de TypeScript tanto no front-end quanto no back-end possibilita a detecção precoce de erros de tipagem em tempo de compilação, reduzindo defeitos de integração e ilustrando o valor prático da tipagem estática em projetos colaborativos. O versionamento com Conventional Commits estabelece um histórico de desenvolvimento semântico, auditável e alinhado com práticas adotadas pela indústria.

14.2 Benefícios Acadêmicos

O projeto integra, em um único artefato funcional, conteúdos de múltiplas disciplinas do curso: Banco de Dados, Desenvolvimento Web, Engenharia de Software, Sistemas de Informação e Testes de Software. Essa convergência demonstra a capacidade da equipe de aplicar conhecimentos de forma transversal e colaborativa, simulando um ciclo completo de desenvolvimento de software.

A documentação produzida, estruturada segundo padrões acadêmicos e de engenharia de software, serve como portfólio técnico para os membros da equipe e como referência para projetos futuros da instituição.

15 Conclusão

O projeto NTPB Dados representa uma iniciativa abrangente de desenvolvimento de software que integra tecnologias modernas, boas práticas de engenharia e fundamentos acadêmicos em uma solução coerente e funcional. A proposta de um dashboard gerencial de vendas com controle de estoque em tempo real demonstra que o conjunto de competências adquiridas ao longo do curso é suficiente para a construção de sistemas com valor comercial real.

A escolha por uma arquitetura em três camadas com comunicação via API REST garantiu ao projeto clareza estrutural e modularidade, permitindo que cada membro da equipe trabalhasse em componentes específicos sem dependências excessivas. A adoção de TypeScript em toda a pilha tecnológica, aliada ao PostgreSQL como banco relacional com integridade referencial, conferiu robustez técnica ao sistema.

Entre os desafios encontrados durante o desenvolvimento, destacam-se a correta implementação da atomicidade nas operações de venda e o alinhamento dos contratos da API entre as camadas front-end e back-end. A resolução dessas questões por meio de transações SQL explícitas e documentação prévia dos endpoints demonstra a maturidade técnica da equipe.

Como perspectivas futuras, o sistema pode ser expandido com módulos de autenticação baseada em JWT, integração com gateways de pagamento reais, funcionalidade de exportação de relatórios em PDF, notificações automáticas por e-mail quando o estoque atingir níveis críticos e deploy em ambiente de nuvem com CI/CD automatizado. Essas extensões consolidariam o NTPB Dados como uma solução de BI de nível profissional.

16 Referências Bibliográficas

CODD, E. F. A relational model of data for large shared data banks. Communications of the ACM, v. 13, n. 6, p. 377-387, jun. 1970.

DATE, C. J. Introdução a sistemas de banco de dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

FEW, Stephen. Information dashboard design: the effective visual communication of data. Sebastopol: O'Reilly Media, 2006.

FIELDING, Roy T. Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado em Ciência da Computação) — University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 2 jun. 2026.

LAUDON, Kenneth C.; LAUDON, Jane P. Sistemas de informação gerenciais. 11. ed. São Paulo: Pearson Education do Brasil, 2014.

MUNZNER, Tamara. Visualization analysis and design. Boca Raton: CRC Press, 2014.

PARMENTER, David. Key performance indicators: developing, implementing, and using winning KPIs. 3. ed. Hoboken: Wiley, 2015.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 7. ed. Rio de Janeiro: GEN LTC, 2020.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Addison Wesley, 2019.

TURBAN, Efraim et al. Business intelligence: a managerial perspective on analytics. 3. ed. Upper Saddle River: Prentice Hall, 2011.

W3SCHOOLS. Node.js Tutorial. Disponível em: <https://www.w3schools.com/nodejs/>. Acesso em: 2 jun. 2026.

REACT DOCUMENTATION. React — The library for web and native user interfaces. Meta Open Source. Disponível em: <https://react.dev>. Acesso em: 2 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 2 jun. 2026.